

Bản trình chiếu: Một vài nghiên cứu về danh sách liên kết theo khối

Su Yu

2008

Nguồn gốc: Một vài nghiên cứu về danh sách liên kết theo khối: bản trình chiếu

IOI China 2008 / Day 2 / Su Yu / slides.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết về danh sách liên kết theo khối. Phần chữ trích xuất được từ PowerPoint cũ chỉ còn các mốc chính: *NOI 2003 Editor*, *NEERC 2003 Key Insertion*, *NOI 2005 Sequence*, *CERC 2007 Sort*, *NOI 2007 Necklace*, các thao tác Move, Insert, Delete, Reverse, và nhận xét về kích thước khối gần $\sqrt{n}$. Vì vậy bản dịch này dùng bài Word đã dịch làm neo nội dung, nhưng giữ giọng của slide: nhấn vào cấu trúc, thao tác và kinh nghiệm chọn khối, không chép lại toàn bộ phân tích dài.

Thông điệp trung tâm là danh sách liên kết theo khối nằm giữa mảng và danh sách liên kết. Mảng định vị nhanh nhưng chèn xóa giữa dãy chậm; danh sách liên kết chèn xóa nhanh nhưng định vị chậm. Nếu mỗi nút của danh sách không chứa một phần tử mà chứa một mảng nhỏ, ta có thể xử lý theo từng khối và cân bằng hai loại chi phí.

2 Cấu trúc cơ bản

Với một dãy dài n , giả sử mỗi khối có dung lượng khoảng x . Khi cần tìm vị trí thứ k , ta quét qua độ dài các khối cho tới khối chứa vị trí đó; chi phí vào khoảng $\mathcal{O}(n/x)$. Khi chèn hoặc xóa trong một khối, phần dịch chuyển cục bộ tốn $\mathcal{O}(x)$. Hai đại lượng cân bằng nhất khi

$$x \approx \sqrt{n},$$

nên thao tác cơ bản thường được ước lượng $\mathcal{O}(\sqrt{n})$.

Slide minh họa các thao tác bằng bài *Editor*. Trình soạn thảo cần hỗ trợ con trỏ, chèn một chuỗi, xóa một đoạn, lấy một đoạn, lùi và tiến con trỏ. Trong danh sách khối, thao tác định vị trả về cặp “khối hiện tại, vị trí tương đối trong khối”. Trước khi chèn hoặc xóa ở giữa khối, ta tách khối tại biên cần thiết, rồi móc thêm hoặc bỏ các khối nằm trọn trong đoạn thao tác.

Nếu tách quá nhiều lần, có thể xuất hiện nhiều khối nhỏ đứng cạnh nhau. Khi đó số khối tăng, làm định vị chậm. Vì vậy cài đặt thực tế cần thao tác gộp: nếu hai khối liên tiếp đều quá thừa và tổng độ dài của chúng không vượt dung lượng cho phép, ghép chúng lại thành một khối. Chính sách này giữ cho số khối không lệch quá xa so với n/x .

3 Mô phỏng mạnh hơn bằng chia khối

Ví dụ *Key Insertion* cho thấy vai trò thực dụng của cấu trúc. Mỗi lệnh đặt một người vào một vị trí; nếu vị trí đã có người, người cũ bị đẩy sang phải, có thể kéo theo cả một đoạn liên tiếp. Nhìn theo dây, đây là thao tác chèn một phần tử vào vị trí L và xóa một ô trống ở cuối đoạn bị đẩy.

Lời giải chính thống dùng DSU và các danh sách phụ khá tinh tế. Slide muốn nhấn mạnh một lựa chọn khác: dùng danh sách khối để mô phỏng trực tiếp đoạn bị đẩy. Đây không nhất thiết là lời giải nhanh nhất, nhưng trong phòng thi nó có lợi thế là dễ nghĩ: ta biến thao tác phức tạp trên đội hình thành chèn xóa trên một dãy.

Khi bài toán yêu cầu nhiều thao tác trên đoạn, danh sách khối mở rộng bằng cách lưu thêm thông tin phụ trong từng khối. Với *Sequence*, mỗi khối có thể lưu tổng, tổng đoạn con lớn nhất, tổng tiền tố/hậu tố tốt nhất, cờ gán đồng loạt và cờ đảo. Khi một đoạn truy vấn phủ trọn một khối, ta dùng ngay thông tin đã lưu; chỉ những khối ở hai biên mới phải đẩy cờ và xử lý từng phần tử.

Với *Sort*, slide ghi rõ thao tác *Reverse* và “minimum in block”. Ý tưởng là lưu giá trị nhỏ nhất của mỗi khối để tìm phần tử cần đưa lên đầu, sau đó đảo tiền tố và loại phần tử đã xử lý. Với *Necklace*, thông tin phụ đổi thành màu hai đầu khối và số đoạn màu liên tiếp trong khối; các thao tác quay, lật, đổi chỗ, tô màu và đếm đoạn màu đều được đưa về xử lý theo khối.

4 Kích thước khối và hiệu năng

Phân tích $\sqrt{n}$ chỉ là điểm xuất phát. Bản trình chiếu có các mốc 6000, 5000, 800 và nhắc các bài *Editor*, *Key Insertion*; chúng khớp với nhận xét trong bài viết: kích thước chạy nhanh nhất thường lớn hơn giá trị lý thuyết. Nguyên nhân là khối không phải lúc nào cũng đầy. Sau nhiều lần tách và gộp, lượng dữ liệu thật trong mỗi khối thường nhỏ hơn dung lượng tối đa, nên muốn độ dài thật gần $\sqrt{n}$, dung lượng tối đa nên lớn hơn $\sqrt{n}$ một chút.

Dữ liệu thao tác cũng quan trọng. Trong *Editor*, số thao tác chèn và xóa bị giới hạn khá nhỏ, trong khi nhiều lệnh chỉ di chuyển con trỏ hoặc lấy một ký tự. Khi đó chi phí định vị chiếm tỷ trọng lớn hơn chi phí sửa trong khối, nên khối lớn có thể nhanh hơn dự đoán. Nếu cài thêm con trỏ hiện tại giống iterator, chi phí định vị giảm, và kích thước tối ưu cũng thay đổi.

Một kinh nghiệm khác là cấp phát động không nên nằm trong vòng lặp nóng. Do tách và gộp diễn ra thường xuyên, các chương trình kèm tài liệu dùng mảng tĩnh và tự quản lý danh sách nút rảnh. Các tệp C++ trong phụ lục nguồn vì vậy không được chép lại ở đây; phần cần giữ cho bản dịch là ý tưởng quản lý khối, cờ lười và thông tin tổng hợp.

5 Giới hạn của cách tiếp cận

Danh sách khối không thay thế cây cân bằng hay cây đoạn. Với cùng một dây động, $\mathcal{O}(\sqrt{n})$ có thể thua xa $\mathcal{O}(\log n)$, và hằng số của cài đặt khối không nhỏ. Trong *Necklace*, nếu nhận ra quay và lật chủ yếu là biến đổi tọa độ, cây đoạn thường gọn và nhanh hơn. Với *Sort*, mô phỏng bằng khối dễ viết hơn một lời giải chuyên biệt, nhưng không nhất thiết đạt tốc độ tốt nhất.

Vì vậy kết luận của slide mang tính cân bằng. Danh sách liên kết theo khối là một công cụ mô phỏng mạnh, dễ mở rộng và tiết kiệm bộ nhớ, đặc biệt hữu ích khi cần chèn xóa trên dây và có thể xử lý nguyên khối. Nhưng nó vẫn đòi hỏi cài đặt cẩn thận: tách, gộp, đẩy cờ, cập nhật thông tin phụ và chọn dung lượng khối đều có thể quyết định bài làm có qua được dữ liệu lớn hay không.